Voyage-AI — Documentation Suite Index & Navigation Guide

Document Reference: `docs/reports/README.pdf` / `docs/reports/README.md`
System Version: 0.2.1 (Phase 2 Verified Baseline)
Publication Date: October 25, 2026
Author: Senior Software Architect & Technical Documentation Specialist
Status: Implementation-Verified Documentation Index

---

1. Overview & Purpose of the Documentation Suite

This documentation suite provides a comprehensive, implementation-verified technical audit of the **Voyage-AI** repository (`g:\Projects\JFS\Voyage-AI`). The platform is an all-in-one travel discovery and itinerary planning application built as a modern decoupled modular monolith uniting a React 18 / TypeScript single-page application with a Java 17 LTS / Spring Boot 4.1.x REST backend, MySQL 8.0 relational persistence foundation, and live Google Gemini AI.

All reports in this suite are grounded strictly in the **current verified source code, executed automated test suites, and empirical host runtime evidence** as of October 25, 2026.

---

2. Master Report Directory & Reading Order

The documentation suite is structured into specialized reports available in both PDF and Markdown formats within `docs/reports/`:

Report File (PDF / MD)	Title	Target Audience	Primary Focus & Coverage
README.md	**Documentation Suite Index**	All Evaluators	High-level map, report abstracts, status definitions, and reading guide.
POST_IMPLEMENTATION_VERIFICATION.md	**Post-Implementation Verification & Final Acceptance**	All Evaluators & Auditors	Traceability matrix, live Google Gemini generation evidence, 128/128 test pass results, and multi-tenant security verification.
implementation-status.md	**Verified Implementation Status**	QA & Product Managers	Evidence-based inventory classifying every component as verified, unverified, partially implemented, or planned.
architecture.md	**System Architecture & Technical Design**	Architects & Engineers	Modular monolith structure, verified technology stack, component diagrams, canonical DB `voyage`, ERD, and API contracts.

Report File (PDF / MD)	Title	Target Audience	Primary Focus & Coverage
testing-and-quality.md	**Testing Strategy & Software Quality**	QA Engineers & Auditors	Multi-tiered test pyramid, verified test execution logs (128/128 passed), database isolation guards, and quality defect analysis.
risks-and-roadmap.md	**Technical Risks & Prioritized Roadmap**	Engineering Leads & Product	Risk register (RSK-01 to RSK-09), P0-P3 prioritized backlog, actionable task breakdowns, and phase exit gates.
project-report.md	**Comprehensive Project Report**	Academic & Project Reviewers	Standalone technical paper: problem statement, scope boundaries, implementation summary, empirical test results, and strategic vision.
methodology.md	**Engineering Methodology & Workflows**	Team Leads & Developers	Phased engineering lifecycle, operational workflows, data pipeline, provider integration rules, and testing layers.

Recommended Reading Sequences:

- **For Technical Evaluators & System Auditors:** `README.md` → `POST_IMPLEMENTATION_VERIFICATION.md` → `architecture.md` → `testing-and-quality.md` → `implementation-status.md`.
- **For Academic Reviewers & Handover Stakeholders:** `README.md` → `POST_IMPLEMENTATION_VERIFICATION.md` → `project-report.md` → `methodology.md` → `risks-and-roadmap.md`.
- **For Contributing Developers:** `README.md` → `architecture.md` → `methodology.md` → `testing-and-quality.md` → `risks-and-roadmap.md`.

---

3. Current Project State & Documentation Scope

3.1 Implemented & Verified Baseline (Phase 2)

- **Frontend Presentation Layer (`frontend/`):** Tactile, accessible single-page application built with React 18, TypeScript, Vite, and Tailwind CSS. Implements responsive navigation, category tabs, server-side catalog pagination (12 items/page), full-text search, 5-tab detail inspection modals with CC-licensed media credits, and live AI Travel Concierge powered by Google Gemini.
- **Backend Service Layer (`backend/`):** Spring Boot 4.1.1 application running on Java 17 LTS on host port `8081`. Exposes `/api/v1/auth/**`, `/api/v1/users/**`, `/api/v1/destinations/**`, `/api/v1/admin/destinations/**`, `/api/v1/trips/**`, `/api/v1/weather`, and `/api/v1/ai/generate` with CORS validation, global exception mapping, and role-based authorization.
- **Relational Persistence Foundation (`backend/` & `infra/`):** MySQL 8.0 canonical schema `voyage` on Docker port 3307 (host port 3306 untouched; legacy `voyage_ai` inactive backup). Flyway versioned migrations (`V0__baseline.sql`, `V1__init_schema.sql`, `V2__seed_travel_data.sql`, `V3__user_profile_and_destination_indexes.sql`, `V4__add_user_roles_and_image_licensing.sql`). Core

domain entities (`User` , `Destination` , `TravelListing` , `Trip` , `TripDay` , `ItineraryItem`) fully mapped and indexed.

- **Real Destination Ingestion Pipeline:** Consumes English Wikipedia summaries and MediaWiki Action API for Creative Commons image licensing (`CC-BY-SA 4.0` , `CC0` , `Public Domain`) and photographer credits, guarded by `@PreAuthorize("hasRole('ADMIN')")` .
- **Automated Verification:** 100% of automated test suites verified passing (128 / 128 tests: 83 backend, 11 frontend, 34 Playwright browser & integration).
- **Local Development Launcher:** Windows `start.bat` orchestrator automating service dependency ordering, readiness health checks, browser launch, and non-destructive service reuse.

3.2 Status Classification Key

- **Implemented and verified:** Code exists and verified by passing test execution or live inspection.
- **Implemented but unverified:** Code exists, but execution was not performed in this session.
- **Partially implemented:** Code exists for a portion of the feature, but the end-to-end flow is incomplete.
- **Planned:** Documented architectural target without active implementation code.
- **Deprecated or superseded:** Replaced or eliminated from current architecture.

---

4. Canonical Repository Documentation References

This documentation suite references and consolidates the following canonical project artifacts:
- Architecture.md: High-level architectural overview and boundaries.
- README.md: Repository quick start, prerequisites, and developer setup.
- PRD.md: Product Requirements Document.
- FRD.md: Functional Requirements Document.
- TDD.md: Technical Design Document.
- IMPLEMENTATION_PLAN.md: Phased development roadmap.
- JWT_AUTH_DESIGN.md: Stateless JWT authentication specification.
- DATA_AUTHENTICITY_AND_PROVIDER_PLAN.md: Data provenance and external provider compliance rules.
- PRE_IMPLEMENTATION_AUDIT_v1.1.md: Pre-implementation baseline audit.